

DEPENDENCY PARSING

Unit III — Exam Notes

1. What is Dependency Parsing?

Dependency Parsing is an NLP technique used to analyze the grammatical structure of a sentence by finding relationships between individual words.

One word acts as the **Head**, another word depends on it and is called the **Dependent**. The relationship is represented by a directed edge/arc. The complete structure is called a **Dependency Tree**.

The boy eats an apple.

```
eats
/ \
boy apple
| |
the an
```

Relationships: eats → boy (subject), eats → apple (object), boy → the (determiner), apple → an (determiner).

2. Basic Terms

Term	Meaning
Head	Main/controlling word that determines the grammatical relationship.
Dependent	Word that depends on another word for its grammatical relationship.
Arc	Directed link between a head and a dependent.
Dependency Relation	The grammatical relationship between head and dependent.
Root	The main/root node of the dependency tree.

3. Common Dependency Labels

Label	Meaning	Example
nsubj	Nominal subject	She runs
obj	Object	She eats apples
amod	Adjectival modifier	The red apple
advmod	Adverb modifier	She runs quickly
det	Determiner	the boy
root	Main/root word	runs

4. Dependency Tree

A dependency tree is a directed graph over the words in a sentence.

Properties:

- There is a single designated ROOT node with no incoming arc.
- Except for the root, every vertex has exactly one incoming arc.
- There is a unique path from the root to each vertex.
- Arcs may be labelled to show the type of syntactic relation.

```
ROOT
|
eats
/ \
```

boy apple
| |
the an

5. Constituency vs Dependency Parsing

Constituency Parsing	Dependency Parsing
Groups words into phrases/constituents	Shows head–dependent relationships between individual words
Phrase-oriented	Word-relation-oriented
Question: Which words form a phrase?	Question: Which word is the head of which word?

6. Types of Dependency Parsing

The PPT presents two dependency parsing paradigms:

1. **Transition-Based Dependency Parsing**
2. **Graph-Based Dependency Parsing**

Memory: Transition-based = construct the dependency tree step by step. Graph-based = find the best dependency tree globally.

TYPE 1 — TRANSITION-BASED DEPENDENCY PARSING

7. Meaning

The parser starts in an initial configuration. At each step it chooses one of several transitions/actions to move to a new configuration. Parsing stops at a terminal configuration and the associated dependency tree is returned.

```
Current Configuration
↓
Choose Transition
↓
New Configuration
↓
Choose Transition
↓
Complete Dependency Tree
```

8. Arc-Standard Algorithm

Arc-standard is a simple transition-based dependency parsing algorithm. It is similar to shift-reduce parsing.

Three main transitions: SHIFT, LEFT-ARC, RIGHT-ARC.

9. Configuration

A configuration has three components:

Configuration = Stack + Buffer + Dependency Graph

- **Stack:** words currently being processed.
- **Buffer:** words not yet processed.
- **Dependency Graph:** dependencies constructed so far.

10. Initial and Terminal Configurations

Initial: all words are in the buffer, stack is empty, graph is empty.

```
STACK BUFFER
[ROOT] I booked a flight
```

Terminal: buffer is empty and stack contains a single word.

11. SHIFT

SHIFT pushes the next word in the buffer onto the stack.

Before: STACK [ROOT] BUFFER [I booked]
 SHIFT
 After: STACK [ROOT I] BUFFER [booked]

Remember: SHIFT = Buffer → Stack.

12. LEFT-ARC

LEFT-ARC adds an arc from the topmost stack word to the second-topmost word and pops the second-topmost word.

Stack: ... s2 s1
 LEFT-ARC
 Dependency: s1 → s2
 Remove s2

Example: flight → the (det). The noun flight is the head and the is its determiner.

13. RIGHT-ARC

RIGHT-ARC adds an arc from the second-topmost stack word to the topmost word and pops the topmost word.

Stack: ... s2 s1
 RIGHT-ARC
 Dependency: s2 → s1
 Remove s1

Example: booked → flight (doobj).

14. Easy Rule

Head is on RIGHT → LEFT-ARC

Head is on LEFT → RIGHT-ARC

15. Complete Transition-Based Example

Sentence: "I booked a flight."

Step	Action	State/Dependency	Explanation
1	SHIFT I	Stack: [ROOT, I] Buffer: [booked, a, flight]	Move I from buffer to stack.
2	SHIFT booked	Stack: [ROOT, I, booked] Buffer: [a, flight]	Now I and booked can form a relation.
3	ARC	booked → I (nsubj)	booked is the head; I is its subject. Remove I.
4	SHIFT a	Stack: [ROOT, booked, a] Buffer: [flight]	Move a to stack.
5	SHIFT flight	Stack: [ROOT, booked, a, flight] Buffer: []	Move flight to stack.
6	ARC	a → flight (det)	a is the determiner of flight. Remove a.
7	ARC	flight → booked (doobj)	flight is the object of booked. Remove flight.

Final dependencies:

booked → I nsubj
 booked → flight doobj
 flight → a det

```

ROOT
|
booked
/ \
I flight
/
a

```

16. Transition-Based General Algorithm

1. Initialize configuration: Stack = [ROOT], Buffer = all words, Graph = empty
2. SHIFT words from Buffer to Stack.
3. When a dependency is identified, apply LEFT-ARC or RIGHT-ARC.
4. Remove the dependent from the Stack.
5. Repeat until terminal configuration.

6. Return the dependency graph/tree.

TYPE 2 — GRAPH-BASED DEPENDENCY PARSING

17. Meaning

Graph-based parsing treats the sentence as a directed graph. Words are nodes, possible dependency relations are edges, and every edge receives a score. The goal is to find the highest-scoring valid dependency tree.

18. Graph-Based Implementation — Step by Step

Step 1: Create a node for every word.

Step 2: Add a special ROOT node.

Step 3: Consider all possible head–dependent pairs.

Step 4: Assign a score to every possible edge.

Step 5: Select edges that form one valid dependency tree.

Step 6: Choose the tree with the highest total score.

19. Maximum Spanning Tree (MST)

Graph-based inference can be treated as finding a Maximum Spanning Tree for a weighted directed graph.

The score of a dependency tree is the sum of the scores of its selected arcs:

$$\text{Score}(\text{Tree}) = \sum \text{Score}(\text{head} \rightarrow \text{dependent})$$

The selected edges must form one valid dependency tree without cycles.

20. Graph-Based Example

Sentence: “She likes apples.”

Nodes: ROOT, She, likes, apples

Possible edges include:

ROOT → likes

likes → She

likes → apples

She → likes

apples → likes

Assume the model assigns:

Edge	Score
ROOT → likes	5
likes → She	4
likes → apples	3
apples → likes	1
She → likes	0

Best tree:

ROOT → likes

likes → She

likes → apples

Total score = 5 + 4 + 3 = 12

21. Graph-Based Example with Full Selection

Sentence: “Book me the morning flight.”

Head → Dependent	Score
------------------	-------

ROOT → Book	9
Book → me	8
Book → flight	7
Book → the	2
Book → morning	3
flight → the	9
flight → morning	8
me → flight	1
morning → flight	2

Selected edges:

ROOT → Book 9
 Book → me 8
 Book → flight 7
 flight → the 9
 flight → morning 8

Total = 9 + 8 + 7 + 9 + 8 = 41

These selected edges form the final highest-scoring valid dependency tree.

22. Transition-Based vs Graph-Based

Transition-Based	Graph-Based
Builds tree step-by-step	Finds best tree globally
Stack + Buffer + Graph	Directed graph
SHIFT / LEFT-ARC / RIGHT-ARC	Edge scores
Local decisions	Global optimization

23. Advantages Mentioned in the PPT

Dependency structure is more suitable for free-word-order languages because it represents grammatical relationships between words rather than relying primarily on fixed word positions.

Example:

Ravi badiki velladu
 Ravi → Subject
 badiki → Destination
 velladu → Verb

Badiki Ravi velladu
 Same basic dependencies, different word order.

24. Last-Minute Revision

Dependency Parsing: Finds grammatical relationships between words.

Head: Main word. **Dependent:** Word attached to head. **Arc:** Link. **Root:** Main node.

Two types: Transition-Based and Graph-Based.

Transition-Based: Stack + Buffer + Graph → SHIFT / LEFT-ARC / RIGHT-ARC → step-by-step tree.

Graph-Based: Nodes → ROOT → possible edges → edge scores → valid tree → maximum total score / MST.

Golden comparison: Transition-based = local, step-by-step. Graph-based = global, best-tree search.

Golden arc rule: Head RIGHT → LEFT-ARC; Head LEFT → RIGHT-ARC.